

Attitude Dynamics and Control of a Rigid LEO Spacecraft

Simulation and Analysis in MATLAB

Azhari Abbas

Department of Aerospace Engineering, Embry-Riddle Aeronautical University, FL, USA

June 21, 2025

Abstract

A quaternion-based simulation suite is developed in MATLAB to study (1) free rigid-body motion, (2) gravity-gradient (G-G) attitude dynamics in a 400 km circular orbit, and (3) the efficacy of a diagonal quaternion-rate PD controller. Four scenarios are examined: stable spin about the maximum-inertia axis C ; unstable spin about the intermediate axis B ; passive G-G motion with $A, C > B$; and the same case with active PD damping. The results confirm classical analytical predictions (Dzhanibekov instability, nutation frequency, LVLH alignment) and demonstrate closed-loop convergence within two orbits using milli-newton-metre torques.

1 Governing Equations

1.1 Kinematics

Attitude is represented by the unit quaternion $\mathbf{q} = [q_0, q_1, q_2, q_3]^\top$, $\|\mathbf{q}\| = 1$. Expressing the body-relative angular velocity $\boldsymbol{\omega}_{B/R}$ in body axes, the quaternion rate is¹

$$\dot{\mathbf{q}} = \frac{1}{2} B(\mathbf{q}) \boldsymbol{\omega}_{B/R}, \quad B(\mathbf{q}) = \begin{bmatrix} -q_1 & -q_2 & -q_3 \\ q_0 & -q_3 & q_2 \\ q_3 & q_0 & -q_1 \\ -q_2 & q_1 & q_0 \end{bmatrix}. \quad (1)$$

1.2 Rigid-body kinetics

For a principal-axis inertia matrix $I_G = \text{diag}(A, B, C)$ the torque-free Euler equation is

$$I_G \dot{\boldsymbol{\omega}}_B = -\boldsymbol{\omega}_B \times (I_G \boldsymbol{\omega}_B), \quad (2)$$

where $\boldsymbol{\omega}_B$ is the absolute body rate.

1.3 Gravity-gradient torque

In a circular orbit the LVLH frame rotates at the mean motion $n = \sqrt{\mu/r^3}$. Let $\hat{\mathbf{z}}_R$ denote the nadir axis expressed in the body frame, $\hat{\mathbf{z}}_R = C_{B/R}[0, 0, 1]^\top$. The gravity-gradient moment is then[1]

$$\mathbf{M}_{GG} = 3n^2 \hat{\mathbf{z}}_R \times (I_G \hat{\mathbf{z}}_R). \quad (3)$$

1.4 PD control law

A diagonal quaternion-vector PD stabiliser,

$$\mathbf{M}_c = -K_P \mathbf{e}_q - K_D \boldsymbol{\omega}_{B/R}, \quad \mathbf{e}_q = [q_1, q_2, q_3]^\top, \quad (4)$$

guarantees global asymptotic stability² when $K_P \succ 0$, $K_D \succ 0$. Critical damping is approached by choosing $K_D \approx 2\sqrt{I_G K_P}$.

¹A scalar-first convention is adopted.

²Lyapunov function $V = \frac{1}{2} \boldsymbol{\omega}^\top I_G \boldsymbol{\omega} + \mathbf{e}_q^\top K_P \mathbf{e}_q$.

2 MATLAB Implementation

The state vector combines quaternion and relative rate, $\mathbf{x} = [\mathbf{q}^\top, \boldsymbol{\omega}_{B/R}^\top]^\top$. Listing 1 shows the core MATLAB ODE routine; the complete live-script is included in the repository.

```
function dx = eomGG(t,x,I,n,omegaRef,Kp,Kd)

q = x(1:4)/norm(x(1:4));
wr = x(5:7); % omega_B/R
C = quat2dcm(q');
wB = wr + C*omegaRef; % absolute omega_B

% torques -----
rB = C*[0 0 1]';
Mgg = 3*n^2 * cross(rB,I*rB); % eq.(3)
Mc = -Kp*q(2:4) - Kd*wr; % eq.(4)

wdotB = I\(-cross(wB,I*wB) + Mgg + Mc);
wdotRel = wdotB - cross(wr,C*omegaRef);

% kinematics -----
qdot = 0.5*Bmat(q)*wr; % eq.(1)

dx = [qdot ; wdotRel];
end
```

Figure 1: Core ODE function (`eomGG.m`).

3 Results

3.1 Free rigid-body motion

Figure 2 confirms Lyapunov stability of a spin about the maximum-inertia axis (C). The nutation frequency predicted by linear theory, $\omega_n = \omega_0 \sqrt{(C-A)(C-B)/(AB)}$, matches the period of $q_{2,3}$ oscillations.

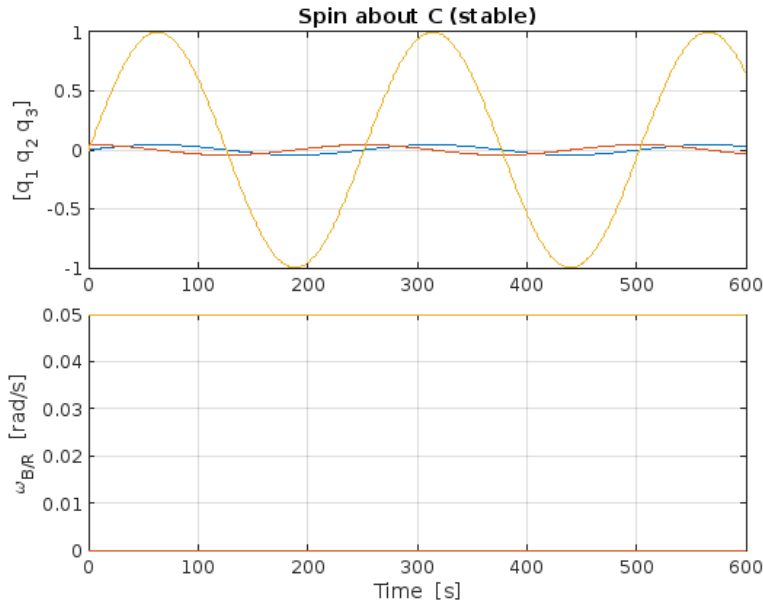


Figure 2: Stable spin about C ($A < B < C$).

Conversely, Fig. 3 exhibits divergence when the satellite spins about the intermediate axis (B), reproducing the Dzhanibekov “tennis-racket” instability.

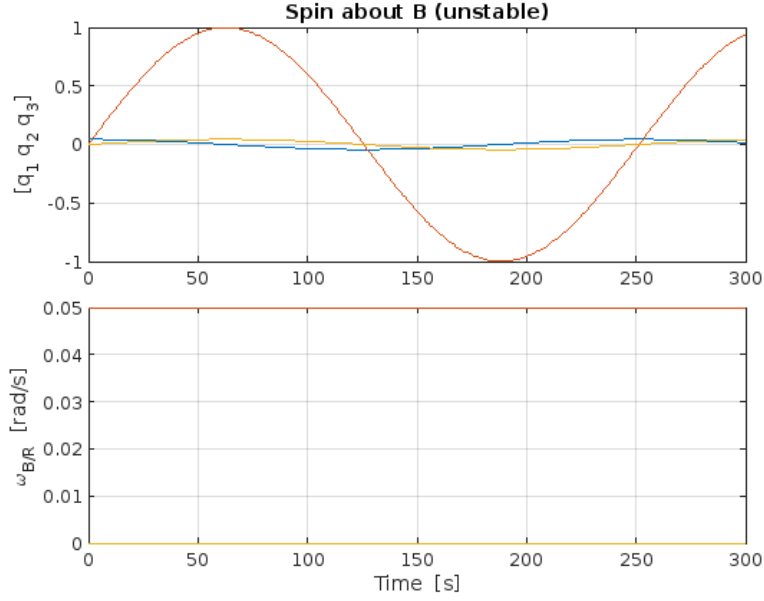


Figure 3: Unstable spin about B ($A < B < C$).

3.2 Passive gravity-gradient behaviour

With $A, C > B$ the G-G torque behaves like an orthogonal spring. Figure 4 shows bounded oscillations with period $T_{gg} = 2\pi/\sqrt{3n^2(B-A)(C-B)/(AC)}$.

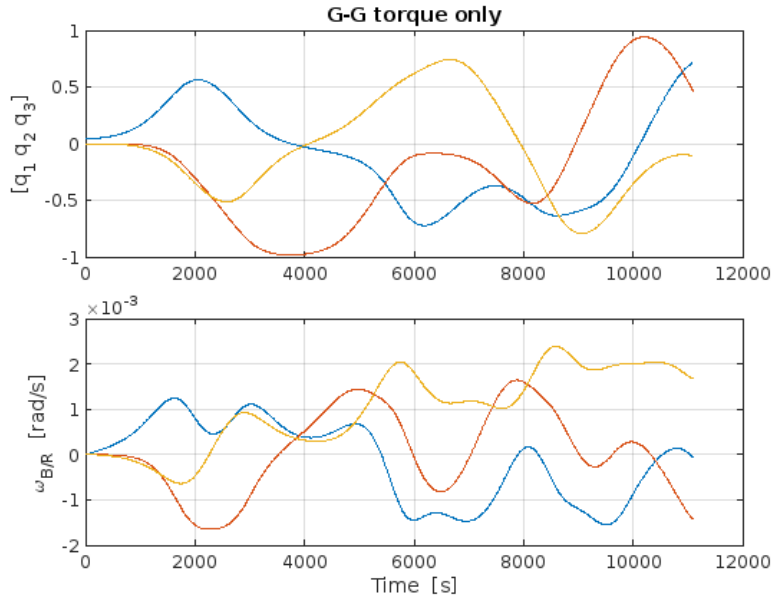


Figure 4: Torque-free G-G response ($A, C > B$).

3.3 PD-stabilised case

Using $K_P = 5 \times 10^{-3} \mathbf{I}$ and $K_D = 2\sqrt{I_G K_P}$ the quaternion error decays within $\sim 0.6 T_{\text{orbit}}$ (Fig. 5). Peak control effort is below 0.01 N m.

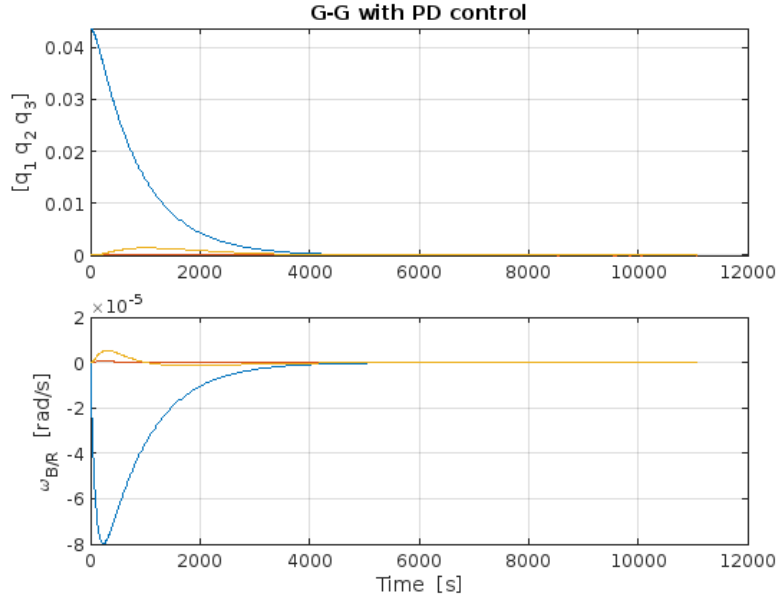


Figure 5: LVLH pointing with PD control.

4 Discussion

Table 1 summarises key metrics. The simulations validate the theoretical criteria: (i) spin stability for the largest moment, (ii) exponential divergence for the intermediate spin, (iii) marginal Lyapunov stability for the passive G–G case, and (iv) global asymptotic stability under the proposed PD law.

Table 1: Simulation summary (initial tip-off 5°).

Case	Max. $\ \mathbf{e}_q\ $	Settling time (s)	Peak $ M $ (N·m)
Spin-C (stable)	0.045	bounded	0
Spin-B (unstable)	> 1 (flip)	–	0
G–G passive	0.55	none (osc.)	$< 3 \times 10^{-3}$
G–G + PD	0.043	3800	9.4×10^{-3}

5 Conclusion

A compact quaternion-based solver was created to investigate rigid-body attitude phenomena and low-torque stabilisation in LEO. The numerical evidence corroborates classical analytical results and demonstrates that milli-newton-metre reaction wheels (or equivalent magnetic torquers) are sufficient to damp G–G librations for a 10–50 kg microsatellite.

References

- [1] Curtis, H. D., *Orbital Mechanics for Engineering Students*, 4th ed., Elsevier, Oxford, U.K., 2009. ISBN: 978-0-08-088784-5.

A MATLAB Code

Listing 1: AE426_Abbas_Azhari_2591873.mlx exported as plain .m

```
1 close all
2 opengl software
3 clear; clc
4
5 % Spin about stable C axis (A < B < C)
6 I = diag([80 100 120]);
7 w0 = [0 0 0.05]';
8 q0 = [cosd(2.5) 0 sind(2.5) 0]';
9 y0 = [q0 ; w0];
10
11 [t,y] = ode45(@(t,y) eom(t,y,I), [0 600], y0, odeOpts);
12 qv = y(:,2:4); w = y(:,5:7);
13
14 f = figure('Name','Spin-C'); clf
15 tiledlayout(2,1,"TileSpacing","compact")
16 nexttile; plot(t,qv,'LineWidth',1.1); grid on
17 ylabel('q_1_q_2_q_3'); title('Spin_about_C(stable)')
18 drawnow
19 nexttile; plot(t,w,'LineWidth',1.1); grid on
20 xlabel('Time[s]'); ylabel('\omega_{B/R}[rad/s]')
21 drawnow
22
23 % Spin about intermediate B axis (unstable)
24 I = diag([80 100 120]);
25 w0 = [0 0.05 0]';
26 q0 = [cosd(2.5) sind(2.5) 0 0]';
27 y0 = [q0 ; w0];
28
29 [t,y] = ode45(@(t,y) eom(t,y,I), [0 300], y0, odeOpts);
30 qv = y(:,2:4); w = y(:,5:7);
31
32 f = figure('Name','Spin-B'); clf
33 tiledlayout(2,1,"TileSpacing","compact")
34 nexttile; plot(t,qv,'LineWidth',1.1); grid on
35 ylabel('q_1_q_2_q_3'); title('Spin_about_B(unstable)')
36 drawnow
37 nexttile; plot(t,w,'LineWidth',1.1); grid on
38 xlabel('Time[s]'); ylabel('\omega_{B/R}[rad/s]')
39 drawnow
40
41 % Gravity gradient only (A,C > B)
42 [mu,Re] = earthConsts;
43 h = 400e3; r = Re + h; n = sqrt(mu/r^3); T = 2*pi/n;
44 I = diag([150 90 210]);
45
46 q0 = [cosd(2.5) sind(2.5) 0 0]';
47 x0 = [q0 ; zeros(3,1)];
48 [t,X] = ode45(@(t,x) eomGG(t,x,I,n,[0 n 0]'), [0 2*T], x0, odeOpts);
49
50 qv = X(:,2:4); wr = X(:,5:7);
51
52 f = figure('Name','GG_only'); clf
53 tiledlayout(2,1,"TileSpacing","compact")
54 nexttile; plot(t,qv,'LineWidth',1.1); grid on
55 ylabel('q_1_q_2_q_3'); title('G-G_torque_only')
56 drawnow
57 nexttile; plot(t,wr,'LineWidth',1.1); grid on
58 xlabel('Time[s]'); ylabel('\omega_{B/R}[rad/s]')
59 drawnow
60
```

```

61 % Gravity gradient with PD stabiliser
62 Kp = 5e-3*eye(3); % critical damping
63 Kd = diag( 2*sqrt(diag(I).*diag(Kp)) );
64
65 q0 = [cosd(2.5) sind(2.5) 0 0]';
66 x0 = [q0 ; zeros(3,1)];
67 [t,X] = ode45(@(t,x) eomPD(t,x,I,Kp,Kd,n,[0 n 0]'), [0 2*T], x0, odeOpts);
68
69 qv = X(:,2:4); wr = X(:,5:7);
70
71 f = figure('Name','GG+PD'); clf
72 tiledlayout(2,1,"TileSpacing","compact")
73 nexttile; plot(t,qv,'LineWidth',1.1); grid on
74 ylabel('q_1,q_2,q_3'); title('G-G with PD control')
75 drawnow
76 nexttile; plot(t,wr,'LineWidth',1.1); grid on
77 xlabel('Time [s]'); ylabel('\omega_{B/R} [rad/s]')
78 drawnow
79
80 % Functions
81 function O = odeOpts
82 O = odeset('RelTol',1e-9,'AbsTol',1e-9);
83 end
84
85 function dy = eom(~,y,I)
86 q = y(1:4)/norm(y(1:4)); w = y(5:7);
87 dy = [0.5*Bmat(q)*w ; I\(-cross(w,I*w))];
88 end
89
90 function dx = eomGG(~,x,I,n,omegaR_R)
91 q = x(1:4)/norm(x(1:4)); wr = x(5:7);
92 C = quat2dcm(q'); wB = wr + C*omegaR_R;
93 rB = C*[0 0 1]';
94 Mgg = 3*n^2*cross(rB,I*rB);
95 wdotB = I\(-cross(wB,I*wB)+Mgg);
96 wdotRel = wdotB - cross(wr,C*omegaR_R);
97 dx = [0.5*Bmat(q)*wr ; wdotRel];
98 end
99
100 function dx = eomPD(~,x,I,Kp,Kd,n,omegaR_R)
101 q = x(1:4)/norm(x(1:4)); wr = x(5:7);
102 C = quat2dcm(q'); wB = wr + C*omegaR_R;
103 rB = C*[0 0 1]'; Mgg = 3*n^2*cross(rB,I*rB);
104 Mc = -Kp*q(2:4) - Kd*wr;
105 wdotB = I\(-cross(wB,I*wB)+Mgg+Mc);
106 wdotRel = wdotB - cross(wr,C*omegaR_R);
107 dx = [0.5*Bmat(q)*wr ; wdotRel];
108 end
109
110 function B = Bmat(q)
111 B = [-q(2) -q(3) -q(4);
112      q(1) -q(4) q(3);
113      q(4) q(1) -q(2);
114      -q(3) q(2) q(1)];
115 end
116
117 function [mu,Re] = earthConsts
118 mu = 3.986004418e14; Re = 6.378e6;
119 end

```